Receitas Médicas como Credenciais Verificáveis (VCs)

Projeto Final Identidades Digitais

Katherine Mosselaar- 163873

ATORES DO SISTEMA

Ideia: criar um sistema seguro e descentralizado de emissão e verificação de receitas médicas, utilizando Smart Contracts para marcar a receita como já usada.

01

Médico (Emissor)

Emite a receita como VC e a envia ao paciente.

02

Paciente (Titular)

Recebe e guarda a receita na carteira digital.

03

Farmácia (Verificador)

Verifica a autenticidade e o status da receita antes da dispensação.

04

Smart Contract & Blockchain

Garante que a receita não possa ser reutilizada.

05

Reguladores

Podem validar os médicos como emissores e farmácias como verificadores confiáveis.

DIAGRAMA DE CONFIANÇA

Médico

(Emissor da VC)

Emite a receita via Traction Schema

Paciente

(Titular da VC)

Envia apresentação da credencial

Farmácia

(Verificador da VC)

Consulta status da receita

Smart Contract

Blockchain (armazena o ID das receitas)

TECNOLOGIAS UTILIZADAS

Traction	Emissão e verificação de VCs.
BC Wallet	Armazenamento da receita pelo paciente.
Node.js (Express)	Backend com webhook e interação com a blockchain.
ethers.js	Biblioteca para interagir com o smart contract.
Ganache	Blockchain privada local.
Remix IDE	Deploy dos smart contracts.

SCHEMA DEFINITION

- Atributos:
  - id_receita
  - data_emissao
  - validade
  - nome_paciente
  - crm
  - medicamento
  - posologia

VERIFICAÇÃO

- Requisição dos atributos:
  - id_receita
  - crm
  - posologia
  - medicamento
- Verifica validade

```
PS C:\Users\kathe\receita-tracker> node server.js

Servidor rodando em http://localhost:3000
Endereço do contrato: 0xc22de56f07849e58a4170280e4058b3f17a2d3e3
Conectado ao: http://127.0.0.1:7545

Processando receita: RX456
Status da receita RX456: Disponível
```

```
PS C:\Users\kathe\receita-tracker> node server.js

Servidor rodando em http://localhost:3000
Endereço do contrato: 0xc22de56f07849e58a4170280e4058b3f17a2d3e3
Conectado ao: http://127.0.0.1:7545

Processando receita: RX456
Status da receita RX456: Já usada
```

ReceitaMedica.sol

1. Declaração do Contrato
2. Armazenamento de Estado
    a. Utilizando o mapping, um dicionário chave-valor (chave=id_receita, valor=true/false)
3. Evento de Log ReceitaRegistrada
    a. Eventos são como logs gravados na blockchain
4. Função pública marcarComoUsada
    a. Verifica se o ID é diferente de NULL
    b. Verifica se já foi usada
    c. Marca como usada e emite evento, ReceitaRegistrada
5. Função de consulta receitaUsada
    a. Retorna true se a receita já foi usada, false caso contrário

ReceitaMedica.sol

Home | ReceitaMedica.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ReceitaMedica {
    mapping(string => bool) private _receitasUsadas;

    event ReceitaRegistrada(string indexed id);

    function marcarComoUsada(string calldata id) external {    infinite gas
        require(bytes(id).length > 0, "ID invalido");
        require(!_receitasUsadas[id], "Receita ja utilizada");

        _receitasUsadas[id] = true;
        emit ReceitaRegistrada(id);
    }

    function receitaUsada(string calldata id) external view returns (bool) {    infinite gas
        return _receitasUsadas[id];
    }
}
```

server.js

1. Importações
    a. Express: framework web para criar a API
    b. Body-parser: faz o parsing do JSON enviado pelo Traction
    c. Ethers.js: conecta com a blockchain (Ganache nesse caso)
2. Configurações do Blockchain (Ganache)
    a. PROVIDER_URL
    b. PRIVATE_KEY da conta
    c. CONTRACT_ADDRESS
3. Definição da ABI
    a. Funções e eventos do contrato
4. Conexão com o contrato via Ethers.js
   ReceitaRegistrada
5. Webhook, integração com Traction
    a. Extrai o campo grupo_receita::id_receita da prova da receita, se não encontrar, retorna erro 400
6. Verificação de duplicidade
7. Verifica se o endereço do contrato está correto
8. Verifica se a receita já foi usada
    a. Se já estiver usada, retorna um HTTP 409 Conflict
9. Marca a receita como usada
    a. Envia uma transação para a blockchain e espera ser minerada
10. Endpoint de saúde

VANTAGEM

- Verificação descentralizada, sem que a farmácia dependa de um servidor central.

DESVANTAGEM

- Farmácia precisa pagar a transação devido ao gás utilizado ao processar o contrato.

PS: Utilizando o mapping não há a questão de vetor fixo, que é o responsável por tornar o processo ineficiente e mais custoso

POSSÍVEL SOLUÇÃO

Criar um contrato com saldo financiado (por exemplo, por um órgão regulador).
As transações de marcarComoUsada usam esse saldo para pagar gás em vez da farmácia.

IPFS

- A farmácia, ao aceitar a receita, marca seu ID como "revogado" em uma lista pública
- Para garantir que este processo seja gratuito a lista deve ser armazenada em IPFS (InterPlanetary File System), com hash imutável
- **Desvantagem:**
  - **pinning**, arquivos são mantidos na rede enquanto alguém os "fixa" localmente → **parcialmente descentralizado**
  - arquivos vizíveis quando o nó que os fixou está online apenas

Referências

- https://docs.soliditylang.org/en/latest/contracts.html
- https://github.com/bcgov/traction
- https://aca-py.org/latest/demo/ACA-Py-Workshop/#lab-4-steps-to-follow
- https://www.researchgate.net/publication/372861132_Blockchain-based_Decentralized_Identity_Management_for_Healthcare_Systems
- https://www.researchgate.net/publication/367841630_Use_of_Blockchain_Technology_for_Electronic_Prescriptions
- https://ipfs.tech/developers/